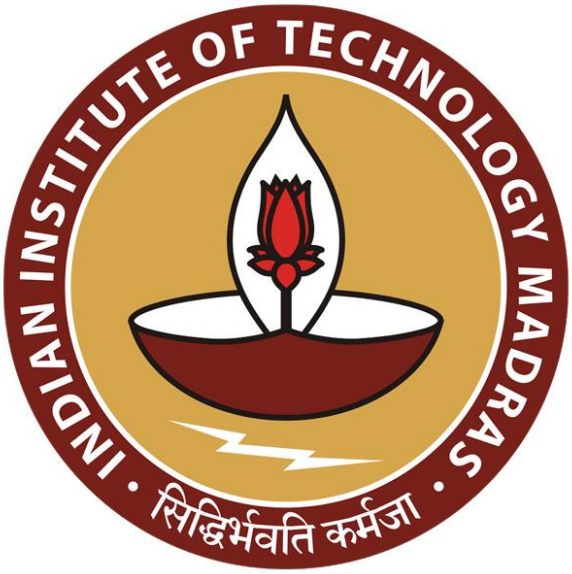


IIT Madras

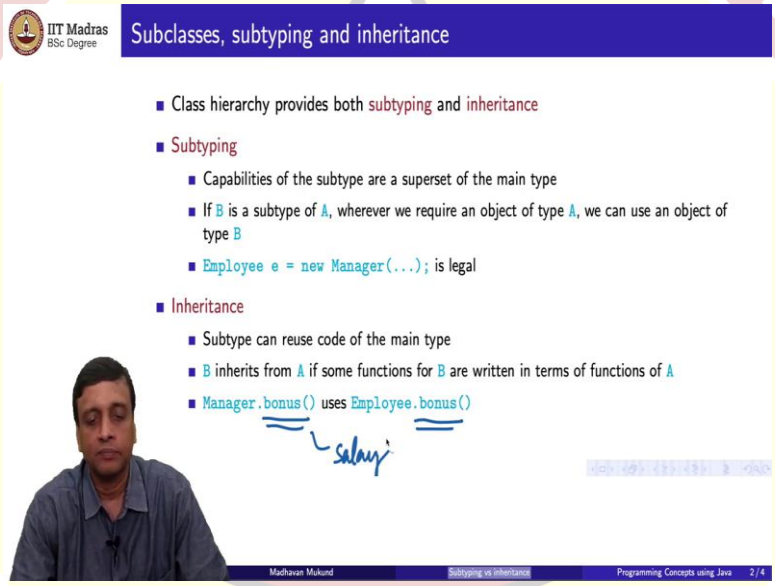
ONLINE DEGREE

Programming Concepts Using Java
Online Degree Programme
B. Sc in Programming and Data Science
Diploma Level
Prof. Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute

Subtyping verses Inheritance

Now that we have seen what the java class hierarchy looks like let us revisit a topic we saw in the first week which is the difference between two concepts in object oriented programming which are called sub typing and inheritance.

(Refer Slide Time: 00:28)



So, the class hierarchy that we have defined in java where we can extend classes in a tree provides both these properties both sub typing and inheritance. So, sub typing effectively captures when one type has more capabilities than another type. So, when we have a sub type it can do everything that the parent type can do and possibly more. So, wherever we expect an object of the parent type, so, if b is a subtype of a and we declare a variable to be of type a then we can always substitute at runtime an object of type b because b has at least the capabilities of a and more.

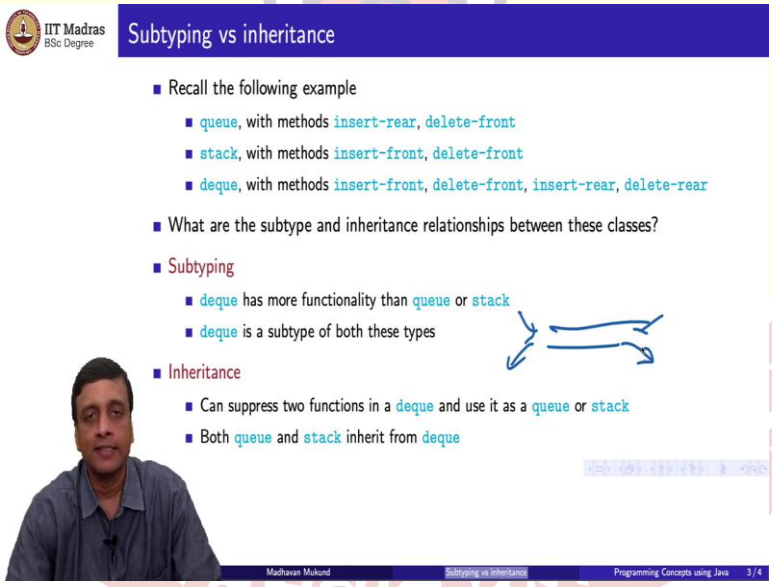
So, any function that we want to invoke on an object of type a will also be available on an object of type b. So, if we saw in our concrete example that we could define manager as a sub type of employee. So, if we define a variable e which is supposed to be of type employee at runtime we can instantiate it to a new manager object this is a legal

instantiation because anything that we would ask an employee type to do can also be performed by manager but remember there is dynamic dispatch and so, on.

So, the same function if it is overridden in the manager will behave according to the runtime behavior of the runtime definition of the object but at the same time nothing that we wish to do on a will be forbidden by b by this manager object. The other part is inheritance. So, inheritance essentially is talking about the sub type reusing code. So, b inherits from a if some functions of b are based on some implementation already written in a.

So, remember that we had defined the bonus function in the manager type and this overrides the employee bonus but one of the complications we had here was this could not access the salary variable because salary was a private variable in employee. So, we did not directly define bonus in manager we rather said that you call the super bonus that is the employees bonus and then multiply that by some factor to get the managers bonus.

(Refer Slide Time: 02:43)



So, the managers bonus is a derived quantity which comes from the employee bonus. So, implicitly the manager bonus is reusing the implementation of the employee bonus. So, to see that these two are different let us look at the example that we have already seen before in the very first week. So, supposing we have these three data structures. So, we have a queue. So, remember that a queue is just a sequence where you enter at one end right and you leave at the other end.

So, this is the normal queue that we are used to when we stand in the line for instance. So, you insert at the rear of the queue you delete from the front of the queue. So, this is a queue right and then a stack is like this normally. So, you think of a stack as having a top. So, you push on the top and you pop from the top but if you want to implement it in the same way. So, then we would typically have both insert and delete come from the front okay and finally we could have a double ended queue or a deque right which allows this at both ends right.

So, this is if it is a deque then we can insert and delete from the front we can insert and delete from the rear. So, these are now three different data structures with different operations for inserting and deleting them. So, clearly we need to be able to identify some sub type and inheritance relation between them because many of these operations are similar. So, if you look at sub typing right what we said was that a sub type should have at least the capabilities of the super type and more.

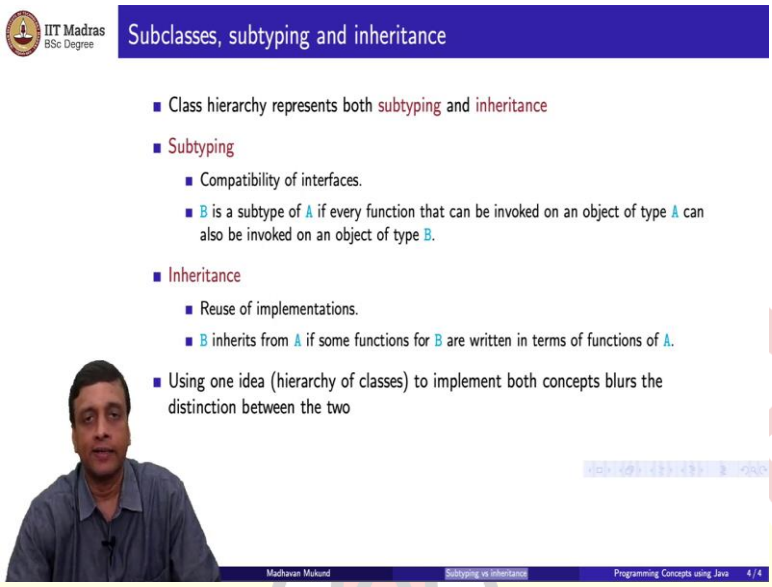
So, q requires to have these two operations well a deque has those two operations and two more. So, a deque is a sub type of queue similarly a stack requires these two operations insert front delete front and again a deque has both of these and more right. So, in both cases deque is a subtype because it has all the capabilities of queue and it has all the capability of stack.

So, anywhere where you want to use a stack and you call a function delete front or delete rear or whatever the thing requires you to do a deque will be able to do it. On the other hand we cannot use a stack directly to implement a deque right because the stack has limited functionality and the same with the queue but on the other hand if we had a deque already. If we had the capability of inserting and deleting at both ends then clearly we can use this implementation to implement either a queue or a stack we just suppress the two functions that we do not want.

So, for a queue we only want insert rear and delete front for a stack we only want insert and delete from the front and nothing from the rear. So, in this sense both stack and queue inherit from deque but deque does not inherit from stack and queue because it cannot reuse anything there it has to actually do it from scratch. So, this way we see that we have two opposite directions right we have these three data types and in the sub

typing sense deque subtypes is a subtype of both stack and queue but in the inheritance sense both queue and stack inherit from deque.

(Refer Slide Time: 05:18)



So, the confusion arises because the class hierarchy is the only way we have of describing the relationship between classes in a language like java or in object oriented programming in general and this hierarchy has to simultaneously represent both sub typing and inheritance. So, in many of the examples that we saw like employee and manager there is no big difference I mean we use the sub type as a more specialized type and it also reuses.

So, inheritance and subtyping go hand in hand but this example with the stack, the deque and the queue show that these two need not be the same. So, remember that sub typing is a compile compatibility with interfaces it talks about what functions you can invoke right. So, every function that can be invoked on a can also be invoked on b when b is a subtype of a whereas inheritance has to do with implementations.

So, you are going to reuse some implementation of a code in the parent type to implement a function in the sub type. So, in this case for example manager uses the employee bonus and. So, b inherits from a some functions for b are written in terms of functions for a. So, using only a single way of describing the relationship between classes that is this class hierarchy where we have something sitting on top of another.

And in particular we have this tree like hierarchy and java this requires us to express both these somewhat independent concepts in terms of a single relationship and therefore sometimes the distinction between the two becomes blurred. So, people will often talk about inheritance and subtyping as the same thing because in all object oriented context is more or less the same thing.

But from a programming language theory point of view these are two different things and it is good to keep it in mind when you talk about programming languages in general.